Neural Scaling Laws.

Bigger models do not improve in fits and starts — their test loss falls along a smooth *power law* that is a straight line on a log–log plot, predictable across many orders of magnitude. This note derives that law, computes it in full, and separates the smooth loss curve from the abrupt jumps of *emergent* downstream abilities.

THE EQUATION

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N} + L_\infty$$

Worked example. $L_\infty = 1.7$, $N_c = 8.8 \times 10^{13}$, $\alpha_N = 0.076$; loss tabulated at $N = 10^6, 10^7, 10^8, 10^9, 10^{10}$.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1 The claim: L is a power law in N

Train a family of transformers of increasing size N (non-embedding parameter count) on enough data, each to convergence, and plot the converged test cross-entropy L. The empirical finding of Kaplan et al. (2020) is that L does not wander: it tracks a *power law* plus a constant floor.

$$\boxed{L(N) \;=\; \left(\frac{N_c}{N}\right)^{\alpha_N} \;+\; L_\infty} \tag{1}$$

Symbol	Meaning	Units / value here
N	model size (non-embedding params)	a count, e.g. 10^6–10^{10}
$L(N)$	converged test loss	nats / token
α_N	scaling *exponent* for N	dimensionless, ≈ 0.076
N_c	scale constant (curvature)	params, 8.8×10^{13}
L_∞	irreducible loss (entropy floor)	nats / token, 1.7

The same functional form holds, with its own exponent, in dataset size D and in compute C:

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} + L_\infty, \qquad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} + L_\infty.$$

Only the constants and exponents differ; the shape is universal.

Loss has two parts. L_∞ is the floor you can never beat — the intrinsic unpredictability of natural language (its entropy), plus modelling error you cannot remove by adding parameters. The term $(N_c/N)^{\alpha_N}$ is the *reducible* loss: the part you buy back by making the model bigger. Doubling N does not halve the reducible loss; it shaves off a fixed *fraction*, which is exactly what "power law" means.

2 Why a power law is a straight line on log–log

Subtract the floor and take logs of the reducible loss $R(N) \equiv L(N) - L_\infty = (N_c/N)^{\alpha_N}$:

$$\log R(N) \;=\; \alpha_N\big(\log N_c - \log N\big) \;=\; \underbrace{\alpha_N \log N_c}_{\text{intercept}} \;-\; \underbrace{\alpha_N}_{\text{slope}} \log N.$$

This is the equation of a straight line in the variables $(\log N,\ \log R)$ with slope $-\alpha_N$. So α_N is literally *how fast loss falls per decade of parameters*: each $10\times$ in N multiplies the reducible loss by $10^{-\alpha_N}$.

$$\frac{R(10N)}{R(N)} = 10^{-\alpha_N} = 10^{-0.076} \approx 0.840 \implies \text{about a } 16\% \text{ drop in reducible loss per decade.} \tag{2}$$

"Predictable over many orders of magnitude" means this line stays straight from $N \sim 10^6$ to $N \sim 10^{10}$ and beyond: fit the slope and intercept on small models, extrapolate the line, and you forecast the loss of a model $10^4\times$ larger before training it.

3 The worked computation

Take $L_\infty = 1.7$, $N_c = 8.8 \times 10^{13}$, $\alpha_N = 0.076$. For each N compute the reducible term $R = (N_c/N)^{0.076}$, then $L = R + L_\infty$. At $N = 10^6$:

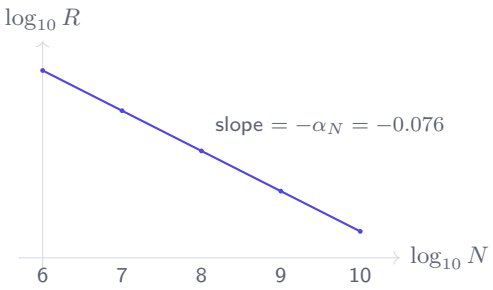
$$\frac{N_c}{N} = \frac{8.8 \times 10^{13}}{10^6} = 8.8 \times 10^7, \quad R = (8.8 \times 10^7)^{0.076} = 10^{\,0.076 \cdot \log_{10}(8.8\times 10^7)} = 10^{0.604} = 4.016,$$

so $L = 4.016 + 1.7 = 5.716$. Repeating across the sweep:

N	$\log_{10} N$	$R = (N_c/N)^{\alpha_N}$	$\log_{10} R$	$L = R + L_\infty$
10^6	6	4.0159	0.6038	5.7159
10^7	7	3.3712	0.5278	5.0712
10^8	8	2.8300	0.4518	4.5300
10^9	9	2.3756	0.3758	4.0756
10^{10}	10	1.9943	0.2998	3.6943

Note the column $\log_{10} R$: it falls by exactly 0.0760 for every $+1$ in $\log_{10} N$. That constant step *is* the straight line — slope $-\alpha_N = -0.076$, intercept $\alpha_N \log_{10} N_c = 0.076 \times 13.944 = 1.060$.

Log–log sketch. Plotting $\log_{10} R$ (vertical) against $\log_{10} N$ (horizontal), the five points lie on one line of slope -0.076:



The same five R values rendered as a heatmap of the reducible loss (darker = larger, i.e. smaller model, more loss left to remove):

	10^6	10^7	10^8	10^9	10^{10}
R	**4.016**	3.371	2.830	2.376	1.994

The exponent α_N is the only thing that matters for the *rate* of improvement. A small exponent (0.076) means brutal economics: each decade of parameters removes only $\sim 16\%$ of the *remaining* reducible loss, so progress demands exponentially more compute. Yet the predictability is the prize — the curve never surprises you.

4 Smooth loss vs. emergent abilities

The power law governs the *pre-training loss*, a smooth average over the whole token distribution. *Downstream* metrics — exact-match accuracy on arithmetic, multi-step reasoning, instruction following — behave differently. As N grows they can sit at chance for many decades and then *jump* sharply once the model crosses a threshold. These are **emergent abilities**.

	Pre-training loss $L(N)$	**Downstream ability**
Shape vs. N	smooth power law	flat $\rightarrow$ sharp jump
Predictable?	yes, across decades	no, jump location is hard to call
Metric	continuous (cross-entropy)	often thresholded (accuracy / exact match)

There is no contradiction: a continuous, smoothly improving probability of emitting the correct token can keep a hard, all-or-nothing metric at zero until the per-token margin finally tips a multi-step task over the line. The loss moved smoothly the whole time; the *measurement* was discontinuous.

5 Properties that matter

1. **Floor.** As $N \rightarrow \infty$, $R \rightarrow 0$ and $L \rightarrow L_\infty$. You cannot beat the entropy floor by scaling parameters alone.

2. **Constant fractional return.** Each decade of N multiplies R by $10^{-\alpha_N}$, independent of where you start — the defining feature of a power law.
3. **Forecasting.** Two cheap small-model points fix slope and intercept; the line then predicts large-model loss before the run.
4. **Three coupled laws.** N, D, and C each have their own exponent; whichever resource is undersized caps the achievable loss (Topic 2).

The straight line is only straight *away* from the limits. Near the irreducible loss L_∞ (very large N) the reducible term is tiny and rounding/floor effects dominate; the curve bends. It also breaks when a *different* resource becomes the bottleneck — e.g. scaling N with a fixed, too-small dataset D: $L(N)$ flattens early because you are now data-limited, not parameter-limited. Extrapolating a clean power law past these regimes over-promises.

6 Where this sits in the track

This topic establishes the single-variable law $L(N)$. **Topic 2** couples N, D and C via $C \approx 6ND$ and asks how to split a fixed compute budget — the question GPT-3 answered one way (scale N) and *Chinchilla* (Module 2.4) later corrected toward ~ 20 tokens per parameter.

A Reproducible (NumPy)

Inputs: $L_\infty = 1.7$, $N_c = 8.8 \times 10^{13}$, $\alpha_N = 0.076$, sweep $N \in \{10^6, \ldots, 10^{10}\}$.

```
import numpy as np
Linf, Nc, alpha = 1.7, 8.8e13, 0.076
N = np.array([1e6, 1e7, 1e8, 1e9, 1e10])
R = (Nc / N)**alpha            # reducible loss
L = R + Linf
for n, r, l in zip(N, R, L):
    print(f"N={n:.0e}  R={r:.4f}  log10R={np.log10(r):.4f}  L={l:.4f}")
# N=1e+06  R=4.0159  log10R=0.6038  L=5.7159
# N=1e+07  R=3.3712  log10R=0.5278  L=5.0712
# N=1e+08  R=2.8300  log10R=0.4518  L=4.5300
# N=1e+09  R=2.3756  log10R=0.3758  L=4.0756
# N=1e+10  R=1.9943  log10R=0.2998  L=3.6943
slope = (np.log10(R[-1]) - np.log10(R[0])) / (np.log10(N[-1]) - np.log10(N[0]))
print("log-log slope =", round(slope, 4))   # log-log slope = -0.076
```